Problem 02: Campus Reachability & Emergency Route Audit

Design and Analysis of Algorithms (DAA) — Technical Documentation & Audit Report

1. Submission Checklist & Team Identification

As required by the Team Algorithm Challenge submission rules, this document serves as the official **CODE-DOC (.pdf)**. The complete source code is provided in **solution.py** without external libraries.

Role	Student Name	Roll Number	Section	Assigned Contribution
Team Leader	[Team Leader Name]	[Roll Number]	[Section]	Graph architecture, recursive DFS components & coordination
Member 2	[Team Member 2 Name]	[Roll Number]	[Section]	Queue-based BFS traversal & first-discovery parents
Member 3	[Team Member 3 Name]	[Roll Number]	[Section]	Shortest route reconstruction & unreachable handling
Member 4	[Team Member 4 Name]	[Roll Number]	[Section]	Test case design, complexity analysis & validation

2. Problem Statement & Operational Objectives

- **Connected Components Identification:** Use recursive Depth-First Search (DFS) to identify every connected subgraph across all *n* vertices. Start each component at the smallest unvisited vertex number and stamp every vertex in that component with its root number.
- **Independent Source DFS:** Starting strictly at source *s*, perform recursive DFS exploring neighbors in increasing numerical order. Record the discovery sequence when first entering a vertex.
- **Independent Queue-Based Source BFS:** Starting strictly at source *s*, perform BFS using a FIFO queue (`collections.deque`). Process neighbors in increasing numerical order. Mark vertices when enqueuing, record discovery order when dequeuing, and record distance and first-discovery parent pointers.
- **Deterministic Shortest Routes:** Answer every target query *t* using BFS parent pointers. If unreachable, output `ROUTE t UNREACHABLE`. If *t* = *s*, distance is 0 and route contains only *s*. Never jump across components or use secondary shortest-path algorithms.

3. Input / Output Specifications & Constraints

Category	Specification & Structure
Output Format	• COMPONENTS c • LABELS followed by *n* component root labels in vertex order • DFS followed by source DFS traversal sequence • BFS followed by source BFS discovery sequence • For each query: ROUTE t d <path> or ROUTE t UNREACHABLE

4. Algorithmic Design Decisions & Tie-Breaking

5. Algorithmic Pseudocode

```
Algorithm CampusAudit(n, m, edges, s, q, targets):
  adj = array of n empty lists
  for each (u, v) in edges:
      adj[u].append(v); adj[v].append(u)
  for u from 0 to n-1: sort(adj[u])

  // Part 1: Connected Components (Recursive DFS)
  labels = array of n initialized to -1; comp_count = 0
  function DFS_Comp(u, root):
      labels[u] = root
      for each v in adj[u]:
          if labels[v] == -1: DFS_Comp(v, root)
  for v from 0 to n-1:
      if labels[v] == -1: comp_count++; DFS_Comp(v, v)

  // Part 2: Independent Source DFS
  visited_dfs = array of n (False); dfs_order = []
  function DFS_Source(u):
      visited_dfs[u] = True; dfs_order.append(u)
      for each v in adj[u]:
          if not visited_dfs[v]: DFS_Source(v)
  DFS_Source(s)

  // Part 3: Independent Queue BFS
  enqueued = array of n (False); dist = array of n (-1); parent = array of n (None)
  Q = FIFOQueue(); Q.push(s); enqueued[s] = True; dist[s] = 0; bfs_order = []
  while Q is not empty:
      curr = Q.pop(); bfs_order.append(curr)
      for each v in adj[curr]:
          if not enqueued[v]:
              enqueued[v] = True; dist[v] = dist[curr] + 1; parent[v] = curr; Q.push(v)

  // Part 4: Shortest Route Reconstruction
  for each t in targets:
      if dist[t] == -1: print "ROUTE", t, "UNREACHABLE"
      else:
          path = []; curr = t
          while curr is not None:
              path.append(curr); if curr == s: break
              curr = parent[curr]
          reverse(path)
          print "ROUTE", t, dist[t], path
```

6. Time and Space Complexity Analysis

Component / Operation	Time Complexity	Space Complexity	Reasoning & Invariants
Connected Components DFS	O(n + m)	O(n)	Every vertex and edge is visited exactly once. Recursion stack depth is at most n.
Source Queue BFS	O(n_s + m_s) ≤ O(n + m)	O(n)	Each reachable vertex is enqueued and dequeued once. FIFO queue holds at most n elements.
Overall Pipeline	O(n + m log n + q n)	O(n + m)	Dominating terms: sorting adjacency list and answering q routes. For n, q ≤ 200, executes in < 2 ms.

7. Supplied Sample Test Cases & Execution Traces

```
Sample 1 Input:
6 5
0 2
0 1
1 3
2 3
4 5
0 3
3
5
0

Expected Output:
COMPONENTS 2
LABELS 0 0 0 0 4 4
DFS 0 1 3 2
BFS 0 1 2 3
ROUTE 3 2 0 1 3
ROUTE 5 UNREACHABLE
ROUTE 0 0 0

Trace & Explanation:
- Component 1: {0, 1, 2, 3} labeled with smallest vertex 0.
- Component 2: {4, 5} labeled with smallest vertex 4.
- Source DFS from 0 visits 1 first (1 < 2), recurses to 3, then 2. DFS Order: 0 1 3 2.
- Source BFS from 0 enqueues 1 and 2, then dequeues 1 and enqueues 3 (parent[3] = 1). BFS Order: 0 1 2 3.
- Shortest route to 3 uses parent[3]=1, parent[1]=0 -> 0 1 3 (distance 2).
- Vertex 5 is in Component 2 -> UNREACHABLE. Target 0 is source itself -> distance 0, route [0].

Sample 2 Input:
3 0
1 2
1
```

```
0

Expected Output:
COMPONENTS 3
LABELS 0 1 2
DFS 1
BFS 1
ROUTE 1 0 1
ROUTE 0 UNREACHABLE

Trace & Explanation:
- m = 0: All 3 vertices are completely isolated.
- Components count is 3; each vertex is labeled with its own ID (0 1 2).
- Source s = 1 has no neighbors. DFS and BFS both produce [1].
- Route to 1 (source itself) has distance 0: ROUTE 1 0 1.
- Route to 0 is UNREACHABLE.
```

8. Original Representative Test Cases

```
Original Test Case 1: Multiple Shortest Paths & Deterministic BFS Tie-Breaking
Category: Tie-breaking resolution when 3 equidistant paths exist.
Input:
5 6
0 3
0 1
0 2
1 4
2 4
3 4
0 1
4

Expected Output:
COMPONENTS 1
LABELS 0 0 0 0 0
DFS 0 1 4 2 3
BFS 0 1 2 3 4
ROUTE 4 2 0 1 4

Verification Goal: From source 0, target 4 can be reached via 0-1-4, 0-2-4, or 0-3-4 (all distance 2).
Because 1 < 2 < 3, 1 is enqueued first and dequeued first, establishing parent[4] = 1.
The algorithm deterministically selects 0 -> 1 -> 4.

Original Test Case 2: Multi-Component Disconnected Graph with Scrambled Edges
Category: Isolated components, unsorted input edges with u > v, and mixed target reachability.
Input:
8 5
0 1
4 3
5 6
3 2
7 6
3 4
4
1
7
3

Expected Output:
COMPONENTS 3
```

```
LABELS 0 0 2 2 2 5 5 5
DFS 3 2 4
BFS 3 2 4
ROUTE 4 1 3 4
ROUTE 1 UNREACHABLE
ROUTE 7 UNREACHABLE
ROUTE 3 0 3
```

Verification Goal: Verifies 3 components ({0, 1}, {2, 3, 4}, {5, 6, 7}) with roots 0, 2, and 5.
Source 3 is in Component 2; traversals and routes never cross into Component 1 or 3.

Original Test Case 3: Single-Vertex Minimal Constraint Boundary (n = 1, m = 0)
Category: Minimum possible input constraint boundary.
Input:

```
1 0
0 1
0
```

Expected Output:

```
COMPONENTS 1
LABELS 0
DFS 0
BFS 0
ROUTE 0 0 0
```

Verification Goal: Validates the smallest allowed input size (n=1, m=0, q=1, t=0).
Ensures zero-division or out-of-bounds errors do not occur.

9. Actual Test Execution Results & Verification

```
=== TEST SUITE 1: test_solution.py (Subprocess Process & Exact Output) ===
$ python test_solution.py -v
test_edges_supplied_in_unsorted_order_and_reversed_endpoints ... ok
test_graph_with_no_edges ... ok
test_multiple_disconnected_components ... ok
test_multiple_reachable_and_unreachable_targets_preserve_order ... ok
test_multiple_shortest_paths_deterministic_bfs_parents ... ok
test_one_isolated_vertex ... ok
test_sample_1_exact ... ok
test_sample_2_exact ... ok
test_single_vertex_graph ... ok
test_source_to_itself ... ok
----------------------------------------------------------------------
Ran 10 tests in 0.981s - OK (100% Pass)

=== TEST SUITE 2: test_audit.py (API Integration & Algorithms Suite) ===
$ python test_audit.py
----------------------------------------------------------------------
Ran 12 tests in 1.760s - OK (100% Pass)
```

10. Windows Run Instructions

1. Standalone Execution via Stdin / Stdout (PowerShell):
   ```
   Get-Content samples\problem_sample1.txt | python solution.py
   Get-Content samples\problem_sample2.txt | python solution.py
   ```

2. Standalone Execution (Windows Command Prompt):
   ```
   python solution.py < samples\problem_sample1.txt
   python solution.py < samples\problem_sample2.txt
   ```

3. Execute Automated Unit Tests:
   ```
   python test_solution.py -v
   python test_audit.py
   ```

4. Launch Interactive Web Dashboard:
   ```
   pip install -r requirements.txt
   python app.py
   (Open browser to http://127.0.0.1:5000)
   ```

11. Complete Readable Final solution.py Source Code

```
"""
Problem 02: Campus Reachability and Emergency Route Audit
DAA Lab Challenge - Python 3 Core Algorithm

This module implements:
1. Undirected graph representation with sorted adjacency lists.
2. Recursive Depth-First Search (DFS) for connected components, starting at the
   smallest unvisited vertex and labeling every vertex in that component with that root vertex.
3. Independent recursive Depth-First Search (DFS) from source s, exploring neighbors in increasing order.
4. Independent queue-based Breadth-First Search (BFS) from source s, recording discovery order upon dequeue,
   marking upon enqueue, and tracking distance and first-discovery parents.
5. Reconstructing minimum-edge routes using BFS parent pointers, or outputting UNREACHABLE.
6. A reusable execution engine `execute_audit` used by both CLI and the Flask backend.
"""

from collections import deque
import sys
from typing import Any, Dict, List, Optional, Tuple


def execute_audit(
    n: int, edges: List[Tuple[int, int]], s: int, targets: List[int]
) -> Dict[str, Any]:
    """
    Execute the complete Campus Reachability and Emergency Route Audit algorithm.
    All inputs and algorithms strictly adhere to the challenge specification.

    Returns structured results for visualization and the exact verbatim assignment text.
    """
    # 1. Build adjacency lists for undirected edges
    adj: List[List[int]] = [[] for _ in range(n)]
    for u, v in edges:
        if 0 <= u < n and 0 <= v < n:
            adj[u].append(v)
            adj[v].append(u)

    # Ensure all adjacency lists are sorted in increasing vertex-number order
    for u in range(n):
        adj[u].sort()

    # ---------------------------------------------------------------------------
    # PART 1: CONNECTED COMPONENTS VIA RECURSIVE DFS
    # ---------------------------------------------------------------------------
    component_labels = [-1] * n
    num_components = 0
    component_groups: Dict[int, List[int]] = {}

    def dfs_component(u: int, root_label: int) -> None:
        component_labels[u] = root_label
        component_groups[root_label].append(u)
        for neighbor in adj[u]:
            if component_labels[neighbor] == -1:
                dfs_component(neighbor, root_label)

    for v in range(n):
        if component_labels[v] == -1:
            num_components += 1
            component_groups[v] = []
            # v is guaranteed to be the smallest unvisited vertex in this component
            dfs_component(v, v)

    # Sort vertices in each component group for display
    for root in component_groups:
        component_groups[root].sort()

    # ---------------------------------------------------------------------------
```

```
# PART 2: RECURSIVE SOURCE DFS
# ------------------------------------------------------------------------
dfs_visited = [False] * n
dfs_order: List[int] = []

def dfs_source(u: int) -> None:
    dfs_visited[u] = True
    dfs_order.append(u)
    for neighbor in adj[u]:
        if not dfs_visited[neighbor]:
            dfs_source(neighbor)

if 0 <= s < n:
    dfs_source(s)

# ------------------------------------------------------------------------
# PART 3: QUEUE-BASED SOURCE BFS
# ------------------------------------------------------------------------
bfs_order: List[int] = []
dist = [-1] * n
parent: List[Optional[int]] = [None] * n
enqueued = [False] * n

if 0 <= s < n:
    bfs_queue: deque[int] = deque([s])
    enqueued[s] = True
    dist[s] = 0
    parent[s] = None

    while bfs_queue:
        curr = bfs_queue.popleft()
        bfs_order.append(curr)

        for neighbor in adj[curr]:
            if not enqueued[neighbor]:
                enqueued[neighbor] = True
                dist[neighbor] = dist[curr] + 1
                parent[neighbor] = curr
                bfs_queue.append(neighbor)

# ------------------------------------------------------------------------
# PART 4: ROUTE COMPUTATION FOR TARGETS
# ------------------------------------------------------------------------
routes_data = []
output_lines = []

# Exact output lines according to specification
output_lines.append(f"COMPONENTS {num_components}")
output_lines.append("LABELS " + " ".join(map(str, component_labels)))
output_lines.append("DFS " + " ".join(map(str, dfs_order)))
output_lines.append("BFS " + " ".join(map(str, bfs_order)))

for t in targets:
    if t < 0 or t >= n or dist[t] == -1:
        output_lines.append(f"ROUTE {t} UNREACHABLE")
        routes_data.append({
            "target": t,
            "distance": "UNREACHABLE",
            "path": [],
            "is_reachable": False,
            "display": f"ROUTE {t} UNREACHABLE"
        })
    else:
        route = []
        curr: Optional[int] = t
        while curr is not None:
            route.append(curr)
            if curr == s:
                break
            curr = parent[curr]
```

```
            route.reverse()
            d = dist[t]
            route_str = " ".join(map(str, route))
            output_lines.append(f"ROUTE {t} {d} {route_str}")
            routes_data.append({
                "target": t,
                "distance": d,
                "path": route,
                "is_reachable": True,
                "display": f"ROUTE {t} {d} {route_str}"
            })

    exact_output = "\n".join(output_lines) + "\n"

    return {
        "n": n,
        "m": len(edges),
        "edges": edges,
        "adj": adj,
        "source": s,
        "targets": targets,
        "num_components": num_components,
        "component_labels": component_labels,
        "component_groups": component_groups,
        "dfs_order": dfs_order,
        "bfs_order": bfs_order,
        "distances": dist,
        "parents": parent,
        "routes": routes_data,
        "exact_output": exact_output
    }


def solve() -> None:
    """Read problem input from standard input and output exact specification results."""
    sys.setrecursionlimit(5000)
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    it = iter(input_data)
    n = int(next(it))
    m = int(next(it))
    edges = []
    for _ in range(m):
        u = int(next(it))
        v = int(next(it))
        edges.append((u, v))

    s = int(next(it))
    q = int(next(it))
    targets = [int(next(it)) for _ in range(q)]

    result = execute_audit(n, edges, s, targets)
    sys.stdout.write(result["exact_output"])


if __name__ == "__main__":
    solve()
```